

Q. Explain Operations on 1-D & 2D Arrays/Lists.
Ans.

A 1-D array is a linear data structure that stores elements of the same type in contiguous memory locations, accessed using a single index.

Common Operations

1. Traversal

Visiting and processing each element of the array exactly once.

Example: Printing all elements

Logic: Start from index 0 to n-1

2. Insertion

Adding a new element at a specific position.

Steps:

1. Check if array has space
2. Shift elements to the right from the given position
3. Insert the new element
4. Increase size of array

3. Deletion

Removing an element from a given position.

Steps:

1. Find the position of the element
2. Shift elements to the left
3. Reduce the array size

4. Searching

Finding the position of an element.

- **Linear Search** – check each element sequentially
- **Binary Search** – only for sorted arrays

6. Sorting

Arranging elements in ascending or descending order.

Common methods:

- Bubble Sort
- Selection Sort
- Insertion Sort

Operations on 2-D Array / List

A 2-D array is a collection of elements arranged in rows and columns, also known as a matrix.

Common Operations

1. Traversal

Accessing each element row-wise or column-wise.

Logic:

Use **nested loops** – outer loop for rows, inner loop for columns.

2. Insertion

Adding an element at a specific row and column.

Steps:

1. Specify row and column index
2. Assign value to that location

3. Deletion

Removing an element from a matrix.

Steps:

1. Identify row and column
2. Replace value with 0 or shift elements as per requirement

4. Searching

Finding an element in the matrix.

Method:

- Sequential search using nested loops
- Optimized search if matrix is sorted

5. Updating

Changing the value of an element.

Example:

matrix[1][2] = 25

Algorithm: Traversal of 1-D Array

Algorithm: Array_Traversal

1. Start
2. Read n
3. Read array elements A[0] to A[n-1]
4. For i = 0 to n - 1
 - Display A[i]
5. Stop

Algorithm: Insertion in 1-D Array

Algorithm: Insert_Element

1. Start
2. Read n
3. Read array elements
4. Read position pos and element x
5. For i = n down to pos
 - $A[i] = A[i - 1]$
6. $A[pos] = x$
7. Increase n by 1
8. Display updated array
9. Stop

Algorithm: Deletion from 1-D Array

Algorithm: Delete_Element

1. Start
2. Read n
3. Read array elements
4. Read position pos
5. For i = pos to n - 2
 - $A[i] = A[i + 1]$
6. Decrease n by 1
7. Display updated array
8. Stop

Algorithm: Linear Search

Algorithm: Linear_Search

1. Start
2. Read n and array elements
3. Read element x
4. For i = 0 to n - 1
 - If $A[i] = x$
 - Display "Element found at position i"
 - Stop
5. Display "Element not found"
6. Stop

Algorithm: Bubble Sort

Algorithm: Bubble_Sort

1. Start
2. Read n and array elements
3. For i = 0 to n - 2
 - For j = 0 to n - i - 2
 - If $A[j] > A[j + 1]$
 - Swap $A[j]$ and $A[j + 1]$
4. Display sorted array
5. Stop

Algorithm: Traversal of 2-D Array

Algorithm: Matrix_Traversal

1. Start
2. Read number of rows r and columns c
3. Read matrix elements
4. For i = 0 to r - 1
 - For j = 0 to c - 1
 - Display $A[i][j]$
5. Stop

Algorithm: Addition of Two Matrices

Algorithm: Matrix_Addition

1. Start
2. Read rows r and columns c
3. Read matrices A and B
4. For i = 0 to r - 1
 - For j = 0 to c - 1
 - $C[i][j] = A[i][j] + B[i][j]$
5. Display matrix C
6. Stop

Algorithm: Transpose of Matrix

Algorithm: Matrix_Transpose

1. Start
2. Read rows r and columns c
3. Read matrix A
4. For i = 0 to r - 1
 - For j = 0 to c - 1
 - $T[j][i] = A[i][j]$
5. Display transpose matrix T
6. Stop

Algorithm: Sum of Array Elements

Algorithm: Sum_Array

1. Start
2. Read n
3. Read array elements
4. Set sum = 0
5. For i = 0 to n - 1
 - sum = sum + $A[i]$
6. Display sum
7. Stop

ARRAY ALGORITHMS

Array Traversal (1-D)

Algorithm: ARRAY_TRAVERSAL

1. Start
2. Read n
3. Read array A[0..n-1]
4. For i = 0 to n-1, display A[i]
5. Stop

Insertion in 1-D Array

Algorithm: ARRAY_INSERT

1. Start
2. Read n, array elements
3. Read pos, x
4. For i = n down to pos
 - A[i] = A[i-1]
5. A[pos] = x
6. Increment n
7. Display array
8. Stop

Deletion from 1-D Array

Algorithm: ARRAY_DELETE

1. Start
2. Read n, array elements
3. Read position pos
4. For i = pos to n-2
 - A[i] = A[i+1]
5. Decrement n
6. Display array
7. Stop

Linear Search

Algorithm: LINEAR_SEARCH

1. Start
2. Read n, array
3. Read element x
4. For i = 0 to n-1
 - If A[i] = x, display position and stop
5. Display "Not Found"
6. Stop

Binary Search

Algorithm: BINARY_SEARCH

1. Start
2. Read sorted array and x
3. Set low = 0, high = n-1
4. While low <= high
 - mid = (low + high)/2
 - If A[mid] = x → found
 - Else if x < A[mid] → high = mid-1
 - Else low = mid+1
5. Stop

SORTING ALGORITHMS

Bubble Sort

Algorithm: BUBBLE_SORT

1. Start
2. Read array
3. For i = 0 to n-2
 - For j = 0 to n-i-2
 - If A[j] > A[j+1] swap
4. Display sorted array
5. Stop

Selection Sort

Algorithm: SELECTION_SORT

1. Start
2. Read array
3. For i = 0 to n-2
 - Find minimum element index
 - Swap with A[i]
4. Display sorted array
5. Stop

Insertion Sort

Algorithm: INSERTION_SORT

1. Start
2. Read array
3. For i = 1 to n-1
 - Insert A[i] at proper position
4. Display array
5. Stop

STACK (Array Implementation)

Push Operation

Algorithm: STACK_PUSH

1. Start
2. If top == max-1, overflow
3. Else
 - top = top + 1
 - stack[top] = x
4. Stop

Pop Operation

Algorithm: STACK_POP

1. Start
2. If top == -1, underflow
3. Else
 - Remove stack[top]
 - top = top - 1
4. Stop

QUEUE (Linear Queue)

Enqueue

Algorithm: QUEUE_INSERT

1. Start
2. If rear == max-1, overflow
3. Else
 - rear = rear + 1
 - queue[rear] = x
4. Stop

Dequeue

Algorithm: QUEUE_DELETE

1. Start
2. If front > rear, underflow
3. Else
 - Remove queue[front]
 - front = front + 1
4. Stop

LINKED LIST

Insertion at Beginning

Algorithm: LL_INSERT_BEGIN

1. Start
2. Create new node N
3. N->next = head
4. head = N
5. Stop

Deletion from Beginning

Algorithm: LL_DELETE_BEGIN

1. Start
2. If head == NULL, empty
3. Else
 - temp = head
 - head = head->next
 - Delete temp
4. Stop

RECURSION

Factorial using Recursion

Algorithm: FACTORIAL

1. Start
2. If n == 0, return 1
3. Else return n * factorial(n-1)
4. Stop

2-D ARRAY / MATRIX

Matrix Traversal

Algorithm: MATRIX_TRAVERSAL

1. Start
2. Read r, c
3. For each row and column display element
4. Stop

Matrix Addition

Algorithm: MATRIX_ADDITION

1. Start
2. Read matrices A and B
3. For all i,j
 - $C[i][j] = A[i][j] + B[i][j]$
4. Display matrix C
5. Stop

Matrix Transpose

Algorithm: MATRIX_TRANSPOSE

1. Start
2. Read matrix A
3. $T[j][i] = A[i][j]$
4. Display T
5. Stop

Q. Discuss string manipulation using arrays.

Ans.

String manipulation using arrays is a common technique in many programming languages—especially in low-level or older languages—because strings are internally represented as arrays of characters

Example: For the string “CAT”

Characters: C A T

Index: 0 1 2

Length calculation:

Count characters until the end

Length = 3

Reversal calculation:

Swap positions

C A T → T A C

Concatenation example:

“CAT” + “S” → C A T S

String operations like length finding, reversing, and concatenation are done using array indexing and calculations, giving direct control over characters.

Q. Explain Arrays and Their Applications

Ans.

Array is a collection of elements of the same data type stored in contiguous memory locations and accessed using an index.

Calculation / Example:

Consider an integer array $A[5] = \{10, 20, 30, 40, 50\}$

- Index: 0 1 2 3 4
- Accessing an element: $A[2] = 30$
- If base address = 1000 and size of int = 4 bytes
Address of $A[2] = 1000 + (2 \times 4) = 1008$

Applications of Arrays:

- Storing and processing lists of data (marks, salaries)
- Used in sorting and searching algorithms
- Representation of matrices and tables
- Used in string storage and manipulation

Q. Define sparse matrix and also give an algorithm to check whether a matrix is sparse matrix or not

Ans.

A sparse matrix is a matrix in which the number of zero elements is greater than the number of non-zero elements.

Condition:

If

Number of zeros > Number of non-zeros

then the matrix is a sparse matrix.

Algorithm to Check Whether a Matrix is Sparse or Not

Step 1: Start

Step 2: Read number of rows m and columns n

Step 3: Read the matrix elements

Step 4: Initialize

- zero = 0
- nonZero = 0

Step 5: For each element $A[i][j]$

- If $A[i][j] == 0$, increment zero
- Else increment nonZero

Step 6:

- If zero > nonZero, print “Matrix is Sparse”
- Else print “Matrix is Not Sparse”

Step 7: Stop

Q. Explain representation of 1D array in the memory with proper example

Ans.

A one-dimensional (1D) array stores elements of the same data type in contiguous (continuous) memory locations. Each element can be accessed using a single index.

Memory Representation

If

- Base address of the array = B
- Size of each element = W bytes
- Index = I

Then the address of the Ith element is calculated as:

$$\text{Address}(A[i]) = B + (i \times W)$$

Example

Let us consider an integer array:

int A[5] = {10, 20, 30, 40, 50}

Assume:

- Base address B = 2000
- Size of int W = 4 bytes

Index	Element	Address Calculation	Memory Address
A[0]	10	2000 + (0 × 4)	2000
A[1]	20	2000 + (1 × 4)	2004
A[2]	30	2000 + (2 × 4)	2008
A[3]	40	2000 + (3 × 4)	2012
A[4]	50	2000 + (4 × 4)	2016

Q. Write an algorithm to delete a node form the end of singly and circular linked list

Ans.

Algorithm to Delete a Node from the End of a Singly Linked List

Algorithm: Delete_End_SLL

Step 1: Start

Step 2: If head == NULL, print “List is Empty” and Stop

Step 3: If head → next == NULL

- Free head
- Set head = NULL
- Stop

Step 4: Set temp = head

Step 5: Traverse the list until temp → next → next != NULL

Step 6: Delete the last node

- free(temp → next)
- temp → next = NULL

Step 7: Stop

Algorithm to Delete a Node from the End of a Circular Linked List

Algorithm: Delete_End_CLL

Step 1: Start

Step 2: If head == NULL, print “List is Empty” and Stop

Step 3: If head → next == head

- Free head
- Set head = NULL
- Stop

Step 4: Set temp = head

Step 5: Traverse until temp → next → next != head

Step 6: Delete the last node

- free(temp → next)
- Set temp → next = head

Step 7: Stop

Q. Diffrentiate between Singly linked list, doubly linked list and circular linked list

Ans.

Feature	Singly Linked List	Doubly Linked List	Circular Linked List
Structure	Each node has data + one link (next)	Each node has data + two links (prev, next)	Last node points back to the first node
Traversal	Can be traversed in one direction only	Can be traversed in both directions	Can be traversed continuously starting from any node
Memory Usage	Requires less memory	Requires more memory due to extra link	Requires no NULL pointer , uses one extra link
Last Node	Last node points to NULL	Last node's next points to NULL	Last node points to first node
Complexity	Simple to implement	More complex	Moderate complexity
Example Use	Stacks, queues	Navigation systems	Circular queues, round-robin scheduling

Q. Explain Operations on a Singly Linked List & Give algorithms.

Ans.

A singly linked list is a linear data structure in which each node contains data and a link (next) pointing to the next node in the list. The last node points to NULL.

1. Algorithm: Insertion at Beginning

Algorithm: Insert_Begin_SLL

Step 1: Start
Step 2: Create a new node newNode
Step 3: Read data into newNode
Step 4: Set newNode → next = head
Step 5: Set head = newNode
Step 6: Stop

2. Algorithm: Insertion at End

Algorithm: Insert_End_SLL

Step 1: Start
Step 2: Create a new node newNode
Step 3: Read data and set newNode → next = NULL
Step 4: If head == NULL, set head = newNode and Stop
Step 5: Set temp = head
Step 6: Traverse while temp → next != NULL
Step 7: Set temp → next = newNode
Step 8: Stop

3. Algorithm: Deletion from Beginning

Algorithm: Delete_Begin_SLL

Step 1: Start
Step 2: If head == NULL, print List Empty and Stop
Step 3: Set temp = head
Step 4: Set head = head → next
Step 5: Free temp
Step 6: Stop

4. Algorithm: Deletion from End

Algorithm: Delete_End_SLL

Step 1: Start
Step 2: If head == NULL, print List Empty and Stop
Step 3: If head → next == NULL
 • Free head, set head = NULL, Stop
Step 4: Set temp = head
Step 5: Traverse while temp → next → next != NULL
Step 6: Free temp → next
Step 7: Set temp → next = NULL
Step 8: Stop

Q. Explain Operations on a Doubly Linked List & give algorithms

Ans. Insertion at Beginning

1. Start
2. Create a new node newNode
3. Read data into newNode
4. Set newNode → prev = NULL
5. Set newNode → next = head
6. If head != NULL, set head → prev = newNode
7. Set head = newNode
8. Stop

Insertion at End

Algorithm: Insert_End_DLL

1. Start
2. Create newNode, set data
3. Set newNode → next = NULL
4. If head == NULL, set newNode → prev = NULL, head = newNode, Stop
5. Set temp = head
6. Traverse while temp → next != NULL
7. Set temp → next = newNode
8. Set newNode → prev = temp
9. Stop

Deletion Operations

(a) Deletion from Beginning

Algorithm: Delete_Begin_DLL

1. Start
2. If head == NULL, print **List Empty** and Stop
3. Set temp = head
4. Set head = head → next
5. If head != NULL, set head → prev = NULL
6. Free temp
7. Stop

(b) Deletion from End

Algorithm: Delete_End_DLL

1. Start
2. If head == NULL, print **List Empty** and Stop
3. Set temp = head
4. Traverse while temp → next != NULL
5. Set temp → prev → next = NULL
6. Free temp
7. Stop

3. Traversal

- Forward Traversal: Using next pointers
- Backward Traversal: Using prev pointers

Algorithm: Traverse_DLL

1. Start
2. Set temp = head
3. While temp != NULL
 - Display temp → data
 - temp = temp → next
4. Stop

Q. Explain Operations on a Circular LL & also give algorithms

Ans.

1. Insertion

(a) Insertion at Beginning

Algorithm: Insert_Begin_CLL

1. Start
2. Create newNode and read data
3. If head == NULL, set newNode → next = newNode, head = newNode, Stop
4. Set temp = head
5. Traverse until temp → next == head
6. Set newNode → next = head
7. Set temp → next = newNode
8. Set head = newNode
9. Stop

(b) Insertion at End

Algorithm: Insert_End_CLL

1. Start
2. Create newNode and read data
3. Set newNode → next = NULL
4. If head == NULL, set newNode → next = newNode, head = newNode, Stop
5. Set temp = head
6. Traverse until temp → next == head
7. Set temp → next = newNode
8. Set newNode → next = head
9. Stop

2. Deletion

(a) Deletion from Beginning

Algorithm: Delete_Begin_CLL

1. Start
2. If head == NULL, print **List Empty** and Stop
3. If head → next == head, free head, set head = NULL, Stop
4. Set temp = head
5. Traverse to last node (temp → next == head)
6. Set temp → next = head → next
7. Free head
8. Set head = temp → next
9. Stop

(b) Deletion from End

Algorithm: Delete_End_CLL

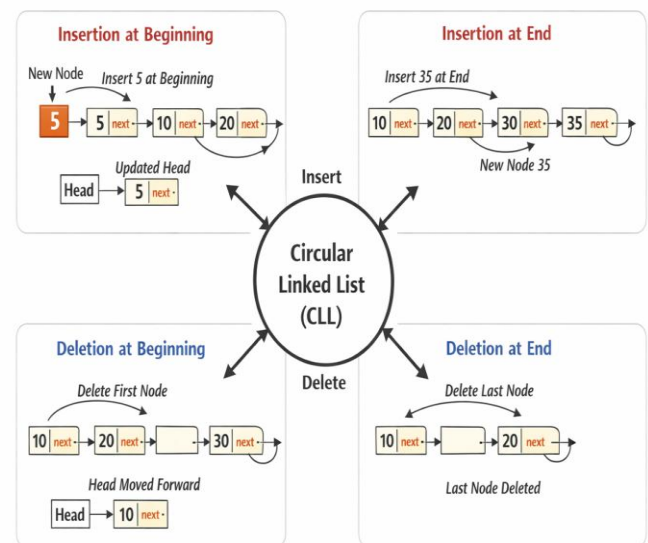
1. Start
2. If head == NULL, print **List Empty** and Stop
3. If head → next == head, free head, set head = NULL, Stop
4. Set temp = head
5. Traverse until temp → next → next == head
6. Free temp → next
7. Set temp → next = head

8. Stop

3. Traversal

Algorithm: Traverse_CLL

1. Start
2. If head == NULL, print **List Empty** and Stop
3. Set temp = head
4. Do
 - Print temp → data
 - temp = temp → next
 While temp != head
5. Stop



Q. Implementation of Stacks Using Arrays and linked lists and also give algorithms

Ans.

A stack is a linear data structure that follows LIFO (Last In, First Out) principle.

- Top element is always the last inserted element.
- Common operations: Push (insert), Pop (delete), Peek (top element), and Display.

1. Implementation Using Arrays

Concept:

- Stack stored in a **1D array**.
- top pointer keeps track of the top element.
- Maximum size is **fixed**.

Operations

(a) Push Operation (Insert an element)

Algorithm: Push_Array

1. Start
2. If top == MAX-1, print **Stack Overflow**, Stop
3. Else, top = top + 1
4. Read element
5. stack[top] = element
6. Stop

(b) Pop Operation (Delete top element)

Algorithm: Pop_Array

1. Start
2. If top == -1, print **Stack Underflow**, Stop
3. Else, print stack[top]
4. top = top - 1
5. Stop

(c) Peek Operation

- Print stack[top] if top != -1

(d) Display Operation

- Traverse from top to 0 and print elements

2. Implementation Using Linked List

Operations

(a) Push Operation

Algorithm: Push_LinkedList

1. Start
2. Create newNode
3. Read data into newNode
4. Set newNode → next = head
5. Set head = newNode
6. Stop

(b) Pop Operation

Algorithm: Pop_LinkedList

1. Start
2. If head == NULL, print **Stack Underflow**, Stop
3. Set temp = head
4. Print temp → data
5. Set head = head → next
6. Free temp
7. Stop

(c) Peek Operation

- Print head → data if head != NULL

(d) Display Operation

- Traverse from head till NULL and print elements

Comparison: Array vs Linked List

Implementation

Feature	Array	Linked List
Memory	Fixed size	Dynamic size
Overflow	Possible	Only if memory full
Implementation	Simple	Requires pointer handling
Access	Index-based	Sequential traversal
Ease of Use	Easier	Slightly complex

Stack follows LIFO (Last In First Out)

Can be implemented using array (fixed size) or linked list (dynamic)

Operations: Push, Pop, Peek, Display

Q. Explain Applications of Stack.

Ans.

A stack works on LIFO (Last In, First Out) principle. It is widely used in situations where last inserted data must be accessed first.

Key Applications:

1. Expression Evaluation
 - Infix → Postfix/Prefix conversion
 - Evaluating arithmetic expressions using stacks
2. Function Call Management
 - Call stack in programming languages to store local variables and return addresses
3. Undo/Redo Operations
 - Used in text editors or software to revert actions
4. Backtracking Algorithms
 - Example: solving maze problems, puzzles, graph traversal (DFS)

Q. Implementation of Queues Using Arrays and linked lists and also give algorithms

Ans.

A **queue** is a linear data structure that follows **FIFO (First In, First Out)** principle.

- **Front:** element to be removed next
- **Rear:** element added last

1.Implementation Using Arrays

Operations

(a) Enqueue (Insert Element)

Algorithm: Enqueue_Array

1. Start
2. If rear == MAX-1, print **Queue Overflow**, Stop
3. Else, if front == -1, set front = 0
4. Increment rear = rear + 1
5. Read element
6. queue[rear] = element
7. Stop

(b) Dequeue (Delete Element)

Algorithm: Dequeue_Array

1. Start
2. If front == -1 or front > rear, print Queue Underflow, Stop
3. Else, print queue[front]
4. Increment front = front + 1
5. Stop

(c) Peek (Front Element)

- Print queue[front] if front != -1

(d) Display Queue

- Traverse from front to rear and print elements

2. Implementation Using Linked List

Concept

- Use a **singly linked list**
- **Front node** acts as front of queue
- **Rear node** keeps track of last node for O(1) insertion

Operations

(a) Enqueue (Insert at Rear)

Algorithm: Enqueue_LinkedList

1. Start
2. Create newNode and read data
3. Set newNode → next = NULL
4. If front == NULL and rear == NULL
 - Set front = rear = newNode
 - Stop
5. Else, rear → next = newNode
6. Set rear = newNode
7. Stop

(b) Dequeue (Delete from Front)

- If queue empty → underflow

- Else, delete front and move front = front → next

Algorithm: Dequeue_LinkedList

1. Start
2. If front == NULL, print **Queue Underflow**, Stop
3. Set temp = front
4. Print temp → data
5. Set front = front → next
6. If front == NULL, set rear = NULL
7. Free temp
8. Stop

(c) Peek (Front Element)

- Print front → data if front != NULL

(d) Display Queue

- Traverse from front till NULL and print each node

Comparison: Array vs Linked List Queue

Feature	Array	Linked List
Memory	Fixed size	Dynamic
Overflow	Possible	Only if memory full
Implementation	Simple	Uses pointers
Time Complexity	O(1) for enqueue/dequeue (if circular)	O(1) for enqueue/dequeue
Access	Index-based	Sequential traversal

Queue follows FIFO

Can be implemented using arrays (fixed size) or linked list (dynamic)

Operations: Enqueue, Dequeue, Peek, Display

Q. Explain Applications of Queues

Ans. A queue works on FIFO (First In, First Out) principle. It is useful in scenarios where first entered data should be processed first.

Key Applications:

1. CPU Scheduling
 - Round-robin scheduling uses queues to manage processes
2. Printer Queue
 - Jobs sent to printer are handled in arrival order
3. Data Buffers
 - Queues store data packets in I/O buffers
4. Breadth-First Search (BFS)
 - Queues are used to traverse trees and graphs

Q. Write an algorithm to perform BFS and DFS with steps.

Ans.

BFS is a graph traversal algorithm that explores all nodes level by level, starting from a source node. It uses a queue to keep track of nodes to visit next.

1. Breadth-First Search (BFS)

BFS is a graph traversal algorithm that explores all nodes level by level, starting from a source node. It uses a queue to keep track of nodes to visit next.

Algorithm: BFS(G, start)

Input: Graph G (vertices and edges), starting node start

Output: Nodes visited in BFS order

Steps:

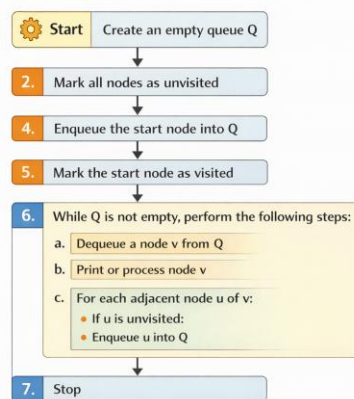
1. Start
2. Create an empty queue Q
3. Mark all nodes as **unvisited**
4. Enqueue the start node into Q and mark it **visited**
5. While Q is not empty:
 - a. Dequeue a node v from Q
 - b. Print v (or process it)
 - c. For each **adjacent node** u of v:
 - If u is unvisited:
 - Mark u as visited
 - Enqueue u into Q
6. Stop

BFS uses queue → FIFO

Good for finding shortest path in unweighted graphs

Diagram:

Algorithm: Breadth-First Search (BFS)



2. Depth-First Search (DFS)

DFS explores a graph **as deep as possible** along each branch before backtracking. It uses a **stack** (or recursion).

Algorithm: DFS(G, start)

Input: Graph G (vertices and edges), starting node start

Output: Nodes visited in DFS order

Steps (Recursive Version):

1. Start
2. Mark start node as **visited**
3. Print start (or process it)
4. For each **adjacent node** v of start:
 - If v is unvisited, call **DFS(G, v)** recursively
5. Stop

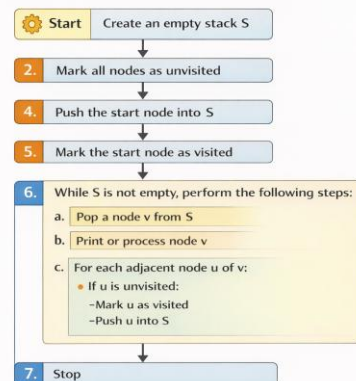
Steps (Using Stack):

1. Start
2. Create an empty stack S
3. Mark all nodes as unvisited
4. Push start node into S and mark it visited
5. While S is not empty:
 - a. Pop a node v from S
 - b. Print v (or process it)
 - c. For each adjacent node u of v:
 - If u is unvisited:
 - Mark u as visited
 - Push u into S
6. Stop

DFS uses stack → LIFO

Useful for cycle detection, topological sorting, connected components

Algorithm: Depth-First Search (DFS)



Q. Explain Binary Search Tree and its operations.

Ans.

A **Binary Search Tree** is a special type of **binary tree** in which:

- Each node contains a **key/value**
- **Left subtree** contains nodes with values **less than** the root
- **Right subtree** contains nodes with values **greater than** the root
- Both left and right subtrees are themselves **BSTs**

Operations on Binary Search Tree

1. Insertion in BST

A new node is inserted at its **correct position** according to BST rules.

Algorithm: Insert_BST(root, key)

1. If root == NULL
 - Create a new node with key
 - Return new node
2. If key < root → data
 - root → left = Insert_BST(root → left, key)
3. Else if key > root → data
 - root → right = Insert_BST(root → right, key)
4. Return root

2. Searching in BST

Search operation compares the key with root and moves left or right.

Algorithm: Search_BST(root, key)

1. If root == NULL or root → data == key
 - Return root
2. If key < root → data
 - Return Search_BST(root → left, key)
3. Else
 - Return Search_BST(root → right, key)

3. Deletion in BST

Three cases arise when deleting a node:

1. **Node with no children (Leaf node)**
2. **Node with one child**
3. **Node with two children** (replace with inorder successor)

Algorithm: Delete_BST(root, key)

1. If root == NULL, return root
2. If key < root → data
 - root → left = Delete_BST(root → left, key)
3. Else if key > root → data
 - root → right = Delete_BST(root → right, key)

4. Else (Node found):

- a. If root → left == NULL
 - Return root → right
- b. If root → right == NULL
 - Return root → left
- c. Else
 - Find inorder successor (minimum value in right subtree)
 - Replace root → data with successor value
 - Delete successor node

5. Return root

Advantages of BST

- Faster search compared to arrays
- Inorder traversal gives **sorted output**
- Dynamic data structure

Time Complexity

Operation	Average Case Worst Case	
	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Q. Explain Tree Traversal. Explain Algorithms for Insertion, Deletion and search.

Ans.

Tree traversal is the process of visiting every node of a tree exactly once in a specific order. Traversal is essential for processing, searching, and printing tree elements.

Tree Operations (Binary Search Tree)

1. Insertion in BST

A node is inserted according to BST property:

- Left child < Root
- Right child > Root

Algorithm: Insert_BST(root, key)

1. If root == NULL
 - Create new node with key
 - Return node
2. If key < root → data
 - root → left = Insert_BST(root → left, key)
3. Else if key > root → data
 - root → right = Insert_BST(root → right, key)
4. Return root

2. Searching in BST

Search compares the key with current node and moves left or right.

Algorithm: Search_BST(root, key)

1. If root == NULL or root → data == key
 - Return root
2. If key < root → data
 - Return Search_BST(root → left, key)
3. Else
 - Return Search_BST(root → right, key)

3. Deletion in BST

Deletion has three cases:

1. **Leaf node**
2. **Node with one child**
3. **Node with two children**

Algorithm: Delete_BST(root, key)

1. If root == NULL, return root
2. If key < root → data
 - root → left = Delete_BST(root → left, key)
3. Else if key > root → data
 - root → right = Delete_BST(root → right, key)
4. Else (Node found):
 - a. If root → left == NULL

- Return root → right
- b. If root → right == NULL
- Return root → left
- c. Else
- Find inorder successor
- Replace root → data
- Delete successor node

5. Return root

1. Inorder Traversal (LNR)

- Visit **Left subtree**, then **Node**, then **Right subtree**
- In a Binary Search Tree, inorder traversal gives **sorted order**

Algorithm: Inorder(root)

1. If root != NULL
2. Inorder(root → left)
3. Print root → data
4. Inorder(root → right)

2. Preorder Traversal (NLR)

- Visit **Node first**, then **Left**, then **Right**
- Used for **creating a copy of tree**

Algorithm: Preorder(root)

1. If root != NULL
2. Print root → data
3. Preorder(root → left)
4. Preorder(root → right)

3. Postorder Traversal (LRN)

- Visit **Left**, then **Right**, then **Node**
- Used for **deleting a tree**

Algorithm: Postorder(root)

1. If root != NULL
2. Postorder(root → left)
3. Postorder(root → right)
4. Print root → data

Tree traversal is used to visit all nodes

Traversal types: Inorder, Preorder, Postorder
BST operations include Insertion, Searching, Deletion

Inorder traversal gives sorted sequence

Q. Explain AVL Tree and its operations. Write algorithms for Insertion, Deletion, Rotations.
Ans.

An AVL Tree (Adelson-Velsky and Landis Tree) is a self-balancing Binary Search Tree (BST) in which:

- For every node,
 Balance Factor (BF) = Height(Left Subtree) – Height(Right Subtree)
- The balance factor of each node is -1, 0, or +1

This property ensures that the tree remains height-balanced, giving $O(\log n)$ time complexity for search, insertion, and deletion.

Algorithms

Algorithm: Insert_AVL(root, key)

1. If root == NULL
 - Create a new node with key and return it
2. If key < root → data
 - root → left = Insert_AVL(root → left, key)
3. Else if key > root → data
 - root → right = Insert_AVL(root → right, key)
4. Update height of root
5. Calculate **Balance Factor (BF)**
6. If BF > 1 and key < root → left → data
 - Perform **Right Rotation (LL)**
7. If BF < -1 and key > root → right → data
 - Perform **Left Rotation (RR)**
8. If BF > 1 and key > root → left → data
 - Perform **Left Rotation** on left child (LR)
 - Perform **Right Rotation**
9. If BF < -1 and key < root → right → data
 - Perform **Right Rotation** on right child (RL)
 - Perform **Left Rotation**
10. Return root

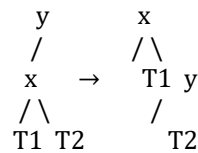
Algorithm: Delete_AVL(root, key)

1. Perform BST deletion to delete the node
2. If root == NULL, return root
3. Update height of root
4. Calculate **Balance Factor**
5. If BF > 1 and BF(left) ≥ 0
 - Perform **Right Rotation (LL)**
6. If BF > 1 and BF(left) < 0
 - Perform **Left Rotation** on left child
 - Perform **Right Rotation (LR)**
7. If BF < -1 and BF(right) ≤ 0
 - Perform **Left Rotation (RR)**

8. If BF < -1 and BF(right) > 0
 - Perform **Right Rotation** on right child
 - Perform **Left Rotation (RL)**
9. Return root

Rotation Algorithms

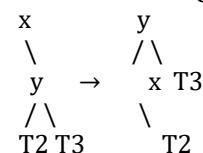
1. Right Rotation (LL Rotation)



Algorithm: Right_Rotate(y)

1. Set x = y → left
2. Set T2 = x → right
3. Set x → right = y
4. Set y → left = T2
5. Update heights of y and x
6. Return x

2. Left Rotation (RR Rotation)



Algorithm: Left_Rotate(x)

1. Set y = x → right
2. Set T2 = y → left
3. Set y → left = x
4. Set x → right = T2
5. Update heights of x and y
6. Return y

Advantages of AVL Tree

- Always balanced → **faster search**
- Guarantees **$O(\log n)$** time complexity
- Better than simple BST in worst cases

Summary

- AVL Tree is a **self-balancing BST**
- Balance Factor $\in \{-1, 0, +1\}$
- Operations: **Insertion, Deletion, Rotations**
- Four rotations: **LL, RR, LR, RL**
- Ensures **logarithmic performance**

Q. Explain Directed and Undirected Graph

Ans.

Graph

A **graph** is a non-linear data structure represented as

$G = (V, E)$

where

- **V** = set of vertices (nodes)
- **E** = set of edges (connections)

1. Undirected Graph

An **undirected graph** is a graph in which **edges have no direction**.

- If there is an edge between vertex **A** and **B**, it is represented as **(A, B)**
- The connection works **both ways**

Characteristics

- No direction on edges
- Symmetric relationship
- If A is connected to B, B is also connected to A

Example

A — B

| |

C — D

Edges: (A, B), (B, D), (D, C), (C, A)

Applications

- Social networks (friendship)
- Road networks (two-way roads)
- Computer networks

2. Directed Graph (Digraph)

A **directed graph** is a graph in which **each edge has a specific direction**.

- Edges are represented as **(A → B)**
- Direction matters

Characteristics

- One-way connections
- Asymmetric relationship
- If A → B, B may not reach A

Example

A → B → C

↑ ↓

E ← D ←

Edges: (A → B), (B → C), (C → D)

Applications

- Web page links
- Traffic flow (one-way roads)
- Task scheduling
- Computer networks

Comparison Table

Feature	Undirected Graph	Directed Graph
Edge Direction	No direction	Direction present
Representation	(A, B)	(A → B)
Relationship	Mutual	One-way
Degree	One degree	In-degree & Out-degree
Example	Friendship	Web links

Conclusion

Undirected graphs represent two-way relationships

Directed graphs represent one-way relationships

Choice of graph depends on the real-world problem

Q. Explain Graph Representations Adjacency matrix, Adjacency List

Ans.

1. Adjacency Matrix

An **adjacency matrix** is a **2-D array** of size **V × V**, where **V** is the number of vertices.

- $matrix[i][j] = 1$ → edge exists between vertex *i* and *j*
- $matrix[i][j] = 0$ → no edge

For **weighted graphs**, the weight is stored instead of 1.

Example (Undirected Graph)

Vertices: A, B, C

Edges: (A, B), (B, C)

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

Advantages

- Simple to understand and implement
- Fast to check if an edge exists (**O(1)**)

Disadvantages

- Wastes memory for sparse graphs
- Requires **V² memory**

2. Adjacency List

An **adjacency list** represents a graph as an **array of linked lists**.

- Each vertex has a list of its adjacent vertices

Example (Same Graph)

A → B

B → A → C

C → B

Advantages

- Efficient use of memory
- Suitable for **sparse graphs**
- Easy to traverse connected vertices

Disadvantages

- Slower edge lookup compared to matrix
- More complex implementation

Comparison Table

Feature	Adjacency Matrix	Adjacency List
Storage	2D Array (V×V)	Array of Lists
Memory Usage	High	Low
Best for	Dense graphs	Sparse graphs
Edge Check	O(1)	O(degree)
Traversal	Less efficient	More efficient

1. Adjacency Matrix is suitable when graph is dense and fast edge lookup is needed

2. Adjacency List is better for large, sparse graphs due to memory efficiency

3. Choice of representation depends on graph size and density

Q. Explain Linear Search & Binary search. Write algorithms to search element using linear and binary search

Ans.

1. Linear Search

Linear search is a simple searching technique in which each element of the list is checked one by one until the required element is found or the list ends.

- Works on sorted or unsorted data
- Time-consuming for large lists

Algorithm: Linear_Search(A, n, key)

1. Start
2. For i = 0 to n - 1
 - If A[i] == key
 - Print "**Element found at position i**"
 - Stop
3. Print "**Element not found**"
4. Stop

Advantages

- Simple and easy to implement
- No need for sorted data

Disadvantages

- Slower for large datasets
- Time complexity **O(n)**

2. Binary Search

Binary search is an efficient searching method that works by repeatedly dividing the sorted list into **two halves**.

- Works **only on sorted data**
- Faster than linear search

Algorithm: Binary_Search(A, n, key)

1. Start
2. Set low = 0, high = n - 1
3. While low ≤ high
 - a. Set mid = (low + high) / 2
 - b. If A[mid] == key
 - Print "**Element found at position mid**"
 - Stop
 - c. If key < A[mid]
 - Set high = mid - 1
 - d. Else
 - Set low = mid + 1
4. Print "**Element not found**"
5. Stop

Advantages

- Very fast searching
- Time complexity **O(log n)**

Disadvantages

- Requires sorted data
- More complex than linear search

Comparison Table

Feature	Linear Search	Binary Search
Data Requirement	Sorted / Unsorted	Sorted only
Time Complexity	O(n)	O(log n)
Method	Sequential	Divide and conquer
Speed	Slow	Fast

Q. Explain Merge sort, Quick sort, Bubble sort. Write algorithms to sort an array using Merge sort, Quick sort & Bubble sort

Ans.

1. Merge Sort

Merge Sort is a **divide and conquer** sorting technique.

- The array is divided into **two halves**
- Each half is sorted recursively
- Sorted halves are **merged**

Features

- Stable sorting algorithm
- Suitable for large datasets
- Requires extra memory

Time Complexity

- Best, Average, Worst: **$O(n \log n)$**

Algorithm: MergeSort(A, low, high)

1. If $low < high$:
2. Find $mid = (low + high) / 2$
3. Call MergeSort(A, low, mid)
4. Call MergeSort(A, mid+1, high)
5. Merge the two sorted halves using Merge(A, low, mid, high)

Algorithm: Merge(A, low, mid, high)

1. Create temporary arrays L and R
2. Copy elements into L and R
3. Compare elements of L and R
4. Place smaller element into original array
5. Copy remaining elements
6. Stop

2. Quick Sort

Quick Sort is a **divide and conquer** algorithm that works by selecting a **pivot element**.

- Elements smaller than pivot go to left
- Elements larger than pivot go to right
- Subarrays are sorted recursively

Features

- Very fast in practice
- In-place sorting
- Not stable

Time Complexity

- Best/Average: **$O(n \log n)$**
- Worst: **$O(n^2)$**

Algorithm: QuickSort(A, low, high)

1. If $low < high$:
2. Find partition index $p = \text{Partition}(A, low, high)$
3. Call QuickSort(A, low, p-1)
4. Call QuickSort(A, p+1, high)

Algorithm: Partition(A, low, high)

1. Choose pivot = $A[high]$

2. Set $i = low - 1$
3. For $j = low$ to $high - 1$:
 - If $A[j] < \text{pivot}$
 - $i++$, swap $A[i]$ and $A[j]$
4. Swap $A[i+1]$ and pivot
5. Return $i+1$

3. Bubble Sort

Bubble Sort is a simple comparison-based technique.

- Adjacent elements are compared
- Larger elements move towards the end
- Repeated until list is sorted

Features

- Simple and easy
- Inefficient for large lists

Time Complexity

- Best: **$O(n)$**
- Average & Worst: **$O(n^2)$**

Algorithm: BubbleSort(A, n)

1. For $i = 0$ to $n-2$:
2. For $j = 0$ to $n-2-i$:
 - If $A[j] > A[j+1]$:
 - Swap $A[j]$ and $A[j+1]$
3. Stop

Comparison Table

Feature	Merge Sort	Quick Sort	Bubble Sort
Method	Divide & Conquer	Divide & Conquer	Comparison
Time Complexity	$O(n \log n)$	$O(n \log n)$ avg	$O(n^2)$
Stability	Stable	Not stable	Stable
Space	Extra memory	In-place	In-place
Efficiency	High	Very high	Low

Conclusion

- **Merge Sort** is best for large data and stable sorting
- **Quick Sort** is fastest in practice
- **Bubble Sort** is suitable for small or already sorted data

Q. Explain Heap. Min heap & Max heap and give algorithms

Ans.

Heap

A **Heap** is a **complete binary tree** that satisfies the **heap property**.

- **Complete Binary Tree:** All levels are completely filled except possibly the last, which is filled from **left to right**
- Heaps are commonly implemented using **arrays**

1. Min Heap

A **Min Heap** is a heap in which:

- The **key of each node is less than or equal to its children**
- The **minimum element** is always at the **root**

Example

```
10
 / \
20 30
 / \
40 50
```

Algorithm: Insert_MinHeap(A, n, key)

1. Increase heap size n
2. Place key at index n
3. While key < parent:
 - Swap with parent
4. Stop

Algorithm: Delete_MinHeap(A, n) (Delete root)

1. Store root in min
2. Replace root with last element
3. Reduce heap size
4. Heapify down to restore heap property
5. Return min

2. Max Heap

A **Max Heap** is a heap in which:

- The **key of each node is greater than or equal to its children**
- The **maximum element** is always at the **root**

Example

```
90
 / \
70 80
 / \
60 50
```

Algorithm: Insert_MaxHeap(A, n, key)

1. Increase heap size n
2. Place key at index n
3. While key > parent:
 - Swap with parent
4. Stop

Algorithm: Delete_MaxHeap(A, n) (Delete root)

1. Store root in max
2. Replace root with last element
3. Reduce heap size
4. Heapify down to restore heap property
5. Return max

Heapify Algorithm (Common for Min & Max Heap)

Algorithm: Heapify(A, i, n)

1. Set largest/smallest = i
2. Compare node with left and right child
3. Swap with largest/smallest child if needed
4. Recursively heapify affected subtree

Time Complexity

Operation Time

Insert	$O(\log n)$
Delete	$O(\log n)$
Search	$O(n)$

Applications of Heap

- Priority Queues
- Heap Sort
- CPU scheduling
- Graph algorithms (Dijkstra, Prim)

Q. Explain Hashing. Explain Hash Tables & Hash Functions with algorithms

Ans.

Hashing is a technique used to map a key value to a fixed index in an array using a hash function, so that search, insertion, and deletion can be performed efficiently.

- Hashing provides fast data access
- Average time complexity is $O(1)$

Hash Table

A **hash table** is a data structure that stores data in an **array format**, where each position is called a **bucket**.

- Data is stored at an index generated by a hash function
- $\text{Index} = h(\text{key})$

Example

If hash table size = 10

Key = 23

Hash function: $h(\text{key}) = \text{key} \% 10$

→ $\text{Index} = 23 \% 10 = 3$

So, key **23** is stored at index **3**

Hash Function

A **hash function** converts a given key into an index of the hash table.

Properties of a Good Hash Function

- Easy to compute
- Uniform distribution of keys
- Minimizes collisions

Common Hash Functions

1. **Division Method:**
 $h(\text{key}) = \text{key} \% \text{table_size}$
2. **Mid-square Method**
3. **Folding Method**

Algorithms

Algorithm: Insert(key) using Chaining

1. Start
2. Compute index using hash function
 $\text{index} = h(\text{key})$
3. If bucket at index is empty
 - Insert key
4. Else
 - Insert key at the end of linked list at that index
5. Stop

Algorithm: Search(key) using Chaining

1. Start
2. Compute $\text{index} = h(\text{key})$
3. Traverse linked list at index
4. If key found
 - Print "Key found"
5. Else
 - Print "Key not found"
6. Stop

Algorithm: Insert(key) using Linear Probing

1. Start
2. Compute $\text{index} = h(\text{key})$
3. If $\text{table}[\text{index}]$ is empty
 - Insert key
4. Else
 - Move to $(\text{index} + 1) \% \text{table_size}$
 - Repeat until empty slot is found
5. Insert key
6. Stop

Algorithm: Search(key) using Linear Probing

1. Start
2. Compute $\text{index} = h(\text{key})$
3. If key found at index
 - Success
4. Else
 - Check next slot using probing sequence
5. If empty slot encountered
 - Search unsuccessful
6. Stop

Advantages of Hashing

- Very fast access ($O(1)$ average)
- Efficient for large datasets

Disadvantages

- Collisions are unavoidable
- Poor hash function reduces performance

Applications of Hashing

- Database indexing
- Password storage
- Symbol tables in compilers
- Caching systems

Conclusion

Hashing maps keys to indices using hash functions

- Hash tables store data efficiently
- Collisions handled using chaining or probing
- Average time complexity is $O(1)$